RESEARCH REPORT · RIFF AUTOMIX ENGINE

From smart crossfades to club-grade transitions: an AutoMix research report for Riff

Structural segmentation, beat and harmonic alignment, transition DSP, industry and academic state of the art, architecture, and a three-phase roadmap, with each technique explained by how it works, what it costs and how it fails.

Date: 31 October 2026 · **Code baseline:** commit `75954bf` on `feat/next-features` (Riff 0.20.0 in progress)
Companion to: `docs/automix-engine-report-2026-10-28.md` (the implementation reference: full derivations of the beat tracker, LR4 coefficients, JSON schemas, state machine). This report doesn't repeat those derivations; it goes beyond them and checks them against the code that has since been built.
Status: research only. No app code was changed for this report.

Contents

1. System context and executive summary
2. Structural segmentation and transition-point selection
3. Tempo, beat-grid and harmonic alignment
4. Transition DSP and typologies
5. Industry benchmarks and academic state of the art
6. System architecture and performance
7. Prioritised implementation roadmap
8. Evaluation protocol
9. References

---

1. System context and executive summary

1.1 The brief's blanks, filled in from Riff's code

The brief left the stack, audio source, capabilities and constraints as placeholders. This is what they are in Riff today (read at commit `75954bf`):

Brief field	Riff today
Tech stack / audio engine	Android 12+, Kotlin, Jetpack Compose, **Media3 ExoPlayer** with a custom audio-processor chain (`EqualizerProcessor` as a TPT state-variable filter bank, `VolumeBoostProcessor` with limiter, `SilenceWatchProcessor`). **ONNX Runtime** ships in 0.20.0 for the Beat This! model. GPL-3.0 licence, which decides which libraries can be bundled.
Audio source	Two kinds. **Local files and downloads**: full random access. **Streamed YouTube Music audio** (Opus/AAC over HTTPS): PCM is available as it plays, but analysis reads *only* from Riff's disk cache once the stream is cached; nothing extra is fetched. So lookahead is full for local songs and opportunistic for streams.

Brief field	Riff today
Current AutoMix capabilities	Settings → Crossfade: **Crossfade / Smart / Automix (beta)**. • **Stage 1 (Smart):** loudness envelope in 100 ms blocks over the last 45 s and first 30 s (`TrackAnalysis.kt`, `SmartTransition.kt`). Finds audible start/end, outro start (level falls below 60 % of the body for good) and intro build-up (80 %). Blend 3–12 s, quiet-intro skip ≤ 8 s. • **Bass swap:** a 150 Hz low shelf, Q 0.7, on each player's equalizer, moved linearly in dB between 0 and −26 dB over 1 s at the blend's midpoint (`bassSwapCuts`). • **Fade law:** equal power, `cos/sin` on each player's `volume` (`crossfadeGains`), written every 16 ms from the control loop. • **Stage 2 (Automix beta):** DSP beat tracker (spectral flux at 11,025 Hz, autocorrelation tempo, Ellis DP beats, bass-weighted downbeat phase, `BeatTracker.kt`). The blend start moves onto a downbeat within 2.5 s, preferring "every fourth downbeat"; B enters on its first downbeat. No time-stretch. • **Stage 4:** Beat This! small0 int8 (4.7 MB ONNX) replaces stage 2's beats when present. On the owner's phone a window costs 3.2–7.3 s and loading 2.0–3.6 s; stalls of minutes were seen under memory pressure, so runs stop after 30 s. • **Two players:** the blend runs on two ExoPlayer instances (main + tail), held within about 120 ms by `CrossfadeTailSynchronizer`. Stage 3 (beat-matched, stretched) hasn't started; its gate is a timing prototype holding the players within 15 ms.
Target hardware / constraints	Mid-range Android phones (the owner's test phone is an I2217). Battery, heat and memory matter more than raw CPU: the stalls above happened with 67 MB free RAM and thermal status 2. APK ceiling raised to 64 MiB for 0.20.0. Nothing may leave the phone (no cloud analysis, privacy policy).

1.2 Executive summary

1. **Two fixable defects in today's blend, found while reading the code.** (a) The bass swap moves *both* shelves linearly in dB, so at its centre both songs' lows are at −13 dB together: the low end drops by about 10 dB for half a second (a "bass hole"). The low band also follows the full-band equal-power fade, so it sags by 3 dB toward the midpoint even outside the swap (figure 2). (b) Stage 2's "four-bar phrase" preference counts every fourth downbeat *from the start of the analysed window*, which begins at an arbitrary point 45 s before the end. Its phase relative to the song's real phrases is effectively random: right only about one time in four. Both are cheap Phase 1 fixes (section 7).
2. **The single biggest structural step is one output with a mixer** (the "capture sink" of the 28 Oct report), not a better model. Sample-accurate alignment, band-split mixing, predictive loudness, stretching and stems all need it. The two-player design can't reliably hold 10–15 ms, and the 31 Oct "+6 dB at handover" bug was a symptom of having two outputs.
3. **Phrase-level cue selection is achievable on the phone** with beat-synchronous self-similarity and novelty, plus a phrase-phase estimator (section 2.3). Its cost is negligible once beats exist. The real limit is the 45 s tail window: 24 bars at 128 BPM, too short to see a 32-bar phrase. Analyse full songs in the background for downloads and local files.
4. **Riff already holds a free vocal-activity signal: synced lyrics.** LRC line times give where vocals are, with no model. Combined with a cheap HPSS/band-energy detector as a cross-check, they make vocal-clash avoidance a Phase 1–2 feature, not a Phase 3 one.
5. **Time-stretching should be small, short and one-sided.** Stretch only the incoming song, only within ±4 % (≤ ±0.68 semitone if done by resampling, so key-lock is required), only during the blend, then ramp back to 1.0 over 8 bars. Rubber Band (GPL, compatible with Riff) is the quality option; WSOLA is enough for drum-led material.

6. **Harmonic mixing should mostly decide *length and type*, not transpose.** Use key compatibility to choose the overlap length and to switch to filter-swap or percussive/echo transitions. Allow ≤ ±1 semitone transposition only on the incoming song, plus a **tuning-offset** correction (cents), which commercial tools rarely mention but which causes audible beating between sustained tones.
7. **Loudness can be predicted, not chased.** Both transition windows are analysed ahead, so the blend's loudness curve can be computed before it plays and a compensating trim lane written offline. That avoids the pumping of a reactive AGC; a true-peak limiter stays as the last safety.
8. **Stems don't need real-time separation.** Separating only the 30–45 s windows ahead of time (while charging) gives most of djay's Neural Mix vocabulary (drum-loop bridges, vocal-muted overlaps) on mid-range Android. Full real-time stems on such phones remain impractical.

> **About the facts and numbers.** Equations and worked numbers in this report were computed for it. Cost figures marked "estimate" are order-of-magnitude reasoning and must be measured on the phone. Product behaviour comes from public reporting (see the 28 Oct report's sources) and is labelled as inferred where it is. The references in section 9 were compiled from the research literature without a fresh web check in this session: titles, venues and years are believed right, but check each before citing it outside the project. Items needing that are marked **[verify]**.

2. Structural segmentation and transition-point selection

A DJ doesn't mix on a beat; they mix on a **phrase**. Electronic and most pop music is built in powers of two: 4 beats to a bar, phrases of 4, 8, 16 and 32 bars. Events (a new layer, a drop, a breakdown, the vocal entry) land on phrase lines. A transition aligned to the beat but not the phrase sounds "correct but wrong": B's chorus arrives two bars into A's phrase.

2.1 The metrical hierarchy and why the window matters

Phrase durations at common tempos (bars × 4 beats × 60 / BPM):

Tempo	1 bar	8 bars	16 bars	32 bars	Bars in Riff's 45 s tail
90 BPM (hip-hop)	2.67 s	21.3 s	42.7 s	85.3 s	16.9
120 BPM (pop)	2.00 s	16.0 s	32.0 s	64.0 s	22.5
128 BPM (house)	1.875 s	15.0 s	30.0 s	60.0 s	24.0
174 BPM (drum & bass)	1.38 s	11.0 s	22.1 s	44.1 s	32.6

The 45 s tail sees one 16-bar phrase at house tempo and never a whole 32-bar one. A phrase-phase estimate from one window therefore rests on one or two phrase boundaries: fragile. Two remedies, both used below:

- **Anchor to strong events** that are nearly always on phrase lines: the outro start, the last downbeat before the audible end, the first downbeat after a breakdown.
- **Analyse the whole song** when it is a local file or download, in the background (section 6.1). Beat This! on a 4-minute song is 8 chunks of 30 s: about 11–32 s CPU on the owner's phone, at measured per-chunk costs. That's fine while charging, too slow on demand.

2.2 Beat-synchronous self-similarity and novelty (Foote)

How	Average features per beat: 12-bin chroma (harmony), 13 MFCCs (timbre), RMS and low-band energy. Build a self-similarity matrix $S_{ij} = \cos(f_i, f_j)$. Correlate a Gaussian-tapered checkerboard kernel (2L beats, L = 16) along the diagonal; peaks of the novelty curve $N(i)$ are boundaries where "before" is self-similar, "after" is self-similar, and the two differ. The 28 Oct report gives the kernel. Snap peaks to the nearest downbeat, then to the phrase grid (2.3).
Cost	For a 75 s analysed window at 128 BPM: about 160 beats, so 25,600 similarities of 38-dim vectors ≈ 1 MFLOP, plus kernel correlation of 32×32 per beat ≈ 0.16 MFLOP. Under 5 ms on a phone. Features are by-products of the STFT the beat tracker already computes.
Failure modes	Gradual changes (filter sweeps, long risers) give flat novelty: no peak. Repetitive EDM where layers enter on top of an unchanged loop gives weak chroma novelty: weight low-band energy and onset density higher. Kernel size sets the scale: L = 16 beats misses 1–2-bar breaks. Window edges (the first and last L beats) can't be scored: pad by mirroring and down-weight them.

2.3 Phrase-phase estimation (the missing piece in stage 2)

Given downbeats $d_0 \ldots d_{K-1}$ and a bar-level evidence score e_k per downbeat, estimate for each phrase length $P \in \{4, 8, 16\}$ the phase ϕ at which phrases start:

```
e_k  = w1·N(d_k)                          Foote novelty sampled at the downbeat
     + w2·|ΔE_low,k|                       bar-to-bar change of low-band energy (kick/bass in or
out)
     + w3·D_chroma,k                       chroma change across the downbeat
     + w4·onsetDensityChange_k
     + β·[k is an anchor]                  outro start, last downbeat before audible end, intro end
score_P(φ) = mean{ e_k : k ≡ φ (mod P) }      φ ∈ {0 … P−1}
φ*_P       = argmax score_P(φ)
conf_P     = (score_P(φ*) − mean_φ score_P) / std_φ score_P
phrase lines = { d_k : k ≡ φ*_P (mod P) }, using the largest P with conf_P ≥ 1.5 (else P = 4, else
none)
```

Defaults: `w = (0.4, 0.3, 0.2, 0.1)`, `β` = the 90th percentile of `e`. Phrases nest, so check consistency: φ^*_8 mod 4 should equal φ^*_4; if not, trust the smaller P.

> **Finding in today's code.** `alignToBeats` prefers `fits.filter { it.index % 4 == 0 }`, where `index` is the position in `outgoing.downbeatsMs`, the downbeats found in the analysed tail. That window starts wherever "45 s before the end" falls, so the 4-bar grid's phase is arbitrary: it matches the song's real phrase lines about one time in four. Replace the index test with `k ≡ φ*₄ (mod 4)` from the estimator above. Lacking confidence, count back from the outro start, which is almost always on a phrase line.

Cost	O(K·P): microseconds.
Failure modes	Pickup bars and odd-length phrases (a 2-bar tag before the chorus, common in pop) shift the phase mid-song: estimate per section, not per window. Songs with a 6/8 feel or tracker downbeat errors give confidently wrong phases; require downbeat confidence ≥ 0.5 and fall back to "beat only". Live drummers drift phrase lengths by a beat: treat as no phrase grid.

2.4 Spectral clustering and structure features (section labels)

How	**Spectral clustering (McFee & Ellis, ISMIR 2014):** build a k-nearest-neighbour recurrence matrix on beat-synchronous chroma/MFCC (k ≈ 2√N, timbre-weighted), add the local-continuity path graph, form the normalised Laplacian, take its first m eigenvectors (m = 4–8) and cluster beats with k-means. Each cluster is a section type (A, B, C…); boundaries are cluster changes. **Structure features (Serrà et al., 2014):** time-lag recurrence plot filtered with a Gaussian; boundaries are its changes. Both give *repetition-based labels* ("this is the chorus again") that the energy rules can't.
Cost	Needs the whole song to be meaningful (repetition is the cue). For a 4 min song at 120 BPM (480 beats): recurrence 230 k pairs, eigendecomposition of a 480×480 symmetric matrix ≈ 10^8 FLOP. Under 0.5 s on a phone; full-song decode dominates (seconds, hardware decoder).
Failure modes	Through-composed songs (no repeats) cluster on timbre only. Choruses that change key in the last repeat split into a new cluster. k-means is random: seed it fixed so a song's labels don't change between runs.

2.5 Deep boundary and function detectors

Model	What it does	Phone cost	Fit for Riff
Boundary CNN on mel-spectrogram (Ullrich, Schlüter & Grill, ISMIR 2014; Grill & Schlüter 2015 with SSLM input)	Frame-wise boundary probability from a ~16 s context	Small (~1 M parameters) **[verify]**; a few ms per second of audio	Train-yourself; no shippable weights with a clear licence known

Model	What it does	Phone cost	Fit for Riff
allin1 (Kim & Nam, WASPAA 2023)	Beats, downbeats, boundaries and labels (intro/verse/chorus/bridge/outro) in one network on *demixed* input	Needs source separation first: heavy (tens of seconds to minutes per song)	Research and offline evaluation only; licence of dependencies **[verify]**
SpecTNT / transformer structure models (2021–2024)	Function labels from spectrograms	Medium to heavy	Evaluation reference
DJ cue-point detectors (Zehren et al., CMJ 2022; object-detection formulation, arXiv 2024)	Predict where DJs actually set cues, trained on DJ annotations (M-DJCUE)	Small	Best target-aligned signal; use its data to **tune** Riff's cue scoring weights

Recommendation: no structure network on the phone in Phases 1–2. Beat-synchronous novelty, phrase phase, energy rules and (for whole songs) spectral clustering cover the decisions AutoMix needs. Learned models are worth it only as a Phase 3 classifier trained on Riff's own data (7.3).

2.6 Scoring cue-out and cue-in candidates

Candidates are phrase-line downbeats. Score each pair `(o ∈ cues(A), i ∈ cues(B))` jointly, because a good transition is a property of the pair, not of each song:

```
J(o, i) = w_s·S_section(o, i)          outro→intro 1.0, breakdown→intro 0.8, outro→drop 0.7,
body→body 0.3
        + w_p·Φ(o)·Φ(i)                phrase strength: 32-bar line 1.0, 16 → 0.9, 8 → 0.75, 4 →
0.5, beat-only 0.2
        − w_v·V(o, i)                  vocal overlap in the blend (2.7), 0…1
        + w_e·E(o, i)                  energy continuity: −|L_A(end of blend) − L_B(start)| / 6
LU, clipped
        − w_d·D(o)                     share of A's remaining music discarded (≤ 15 % allowed,
BitChord rule)
        − w_k·(1 − K(A, B))·len(o, i)  key clash scaled by overlap length (3.4)
choose argmax J subject to: blend length within the type's range; B's skipped intro has no vocals.
```

Starting weights `w = (1, 0.8, 1.5, 0.5, 2, 0.6)`; tune them against human cue choices (M-DJCUE) and Riff's own skip data (8.3). The search is tiny (≈ 10 × 10 candidates).

2.7 Vocal activity detection and clash avoidance

Method	How it works	Cost	Failure modes
Synced lyrics (Riff-specific, Phase 1)	Riff already fetches time-synced lyrics (LRC) and draws gap dots between lines. A line start t_j to the next line start (or a gap marker) gives a vocal interval. `v(t) = 1` inside lines, 0 in gaps ≥ 2 s.	Zero: already downloaded	Line-level only (no word ends); timings can be offset by a different edit (radio/album/extended); instrumental breaks inside a "line"; missing for many songs. Cross-check with a detector and reject lyrics whose offset disagrees by > 1 s.
HPSS + band heuristic	Median-filter HPSS (Fitzgerald 2010) → harmonic part; vocal score from 300–3,400 Hz energy share × (1 − spectral flatness) × f0 salience; smoothed 0.5 s; threshold at the song's 70th percentile (28 Oct report, 2.4).	Two median filters on the existing STFT: ~2–5 ms per second of audio (estimate)	Confuses lead synths, guitars and saxophone with voice; misses whispered or heavily processed vocals; rap over sparse beats scores low on f0 salience.

Method	How it works	Cost	Failure modes
Small CNN singing-voice detector (Schlüter & Grill, ISMIR 2015; Lee, Choi & Nam, ISMIR 2018 review)	Mel patches (~1.6 s) → frame probability; trained with pitch-shift and time-stretch augmentation	< 1 MB int8; a few ms per second of audio (estimate)	Same synth/voice confusion, less often; needs own training (licences of public weights unclear) **[verify]**
Separated vocal stem RMS (Phase 3)	Vocal stem from separation (7.3); active where its short-term level is within 20 dB of its own peak	Free once stems exist	Bleed: reverb tails and backing vocals read as "active"; use a higher threshold.

Clash measure. With aligned vocal curves over the planned overlap $[t_0, t_1]$ and the gains each deck will have:

```
V = (1 / beat) · ∫ min( g_A(t)·v_A(t), g_B(t)·v_B(t) ) dt      in beats of "both singing audibly"
Rule: V ≤ 1 beat.  Else, in order: move the cue pair (2.6) → shorten the overlap → duck B's 300–
3,400 Hz
by −9 dB (peaking EQ, Q 0.7) until A's vocal ends → (with stems) mute B's vocal stem until A's
ends.
```

Using `min(g·v)` rather than the product allows the classic move of A's last vocal phrase fading under B's instrumental intro; that isn't a clash.

3. Tempo, beat-grid and harmonic alignment

3.1 Drifting and non-quantised tempo: align beat *phase*, not BPM

A single BPM is a lossy summary. Live-played and older records drift 1–4 % within a song, and a BPM-matched blend drifts out of phase within a few bars: at 120 BPM, a 0.5 % error accumulates 10 ms per bar, an audible flam after 2 bars. The robust formulation works on **beat-phase functions**.

```
β_A(t)  : beat index of A at its content time t, by linear interpolation between tracked beats
          (β_A(b_k) = k), so tempo can vary per beat
τ_B(t)  : B's content position when the output plays A at t.  Its rate is r(t) = dτ_B/dt.
Goal    : β_B(τ_B(t)) = β_A(t) + k₀          (B's beat k₀ lands on A's beat 0, and stays locked)
Solution: r(t) = (dβ_A/dt) / (dβ_B/dt) evaluated at τ_B(t)   =  T_B,local / T_A,local
          (ratio of B's local beat period to A's, from the beat lists; piecewise constant per
beat)
```

With Beat This!, beat times aren't quantised to a grid (it has no DBN), so they are already a warp map: use them directly, with two guards:

- **Smooth the ratio:** a beat-time jitter of ±10 ms at 128 BPM is ±2 % of a beat period. Fit a local line through 8 beats (least squares) before taking periods, or the stretcher's rate wobbles audibly.
- **Closed-loop correction** in the mixer: measure phase error `e = β_B(τ_B) − β_A(t) − k₀` once per beat (from content positions, not audio) and correct with `r ← r_ff · (1 + K_p·e)`, K_p = 0.02 per beat of error, rate correction clamped to ±0.5 %. With the single-output mixer (6.2) there's no clock drift; the loop only absorbs tracker error.

Cost	Negligible (one interpolation per block); the stretcher (3.2) pays for the varying rate.
Failure modes	Octave errors (one song tracked at double time): fold ratios to {½, ⅔, 1, 3/2, 2} first. Swing and triplet feels: beat *positions* are right but subdivision differs; keep overlaps with clashing hi-hat patterns short. Tracker skips a beat: the phase function jumps by one; detect inter-beat intervals outside 0.7–1.3× the local median and repair before warping.

3.2 Time-stretching: algorithms compared

Requirement: tempo change of at most ±4 % (that's a pitch shift of ±0.68 semitone if done by resampling, plainly out of tune), applied to one deck, for 8–32 bars, with sharp transients (kicks) preserved.

Method	How it works	Cost (stereo 48 kHz)	Artefacts at ±4 %	Fit
Resampling ("vinyl")	Variable-rate interpolation; pitch moves with tempo	Tiny	None except pitch change ($12 \cdot \log_2 r$ semitones)	Only for effects (brake, 3.5) and tuning-offset fixes below 0.3 %
WSOLA (Verhelst & Roelands 1993); SoundTouch	Overlap-add of ~20–40 ms grains, each shifted ±5 ms to the position most similar to the natural continuation	~1–3 % of a core (estimate); latency 25–100 ms	Very low on drums; slight warble on sustained chords and vocals; transient doubling at larger ratios	Good for drum-led genres; SoundTouch is LGPL-2.1

Method	How it works	Cost (stereo 48 kHz)	Artefacts at ±4 %	Fit
Sonic (Media3's `SonicAudioProcessor`)	Pitch-period overlap-add, tuned for speech	Low	Graininess on sustained music; slowing down sounds metallic	Already in the app; acceptable for speed-ups only
Phase vocoder + identity phase locking (Laroche & Dolson 1999)	STFT; per-peak phase advance by instantaneous frequency; bins locked to their peak	FFT 2048–4096 per hop: ~3–8 % of a core (estimate); latency ≥ 1 frame (43–85 ms)	Low on tonal music; transient smearing ("phasiness" on drums) unless phases reset at onsets	Basis of the best stretchers
PV with transient preservation (Röbel 2003) or HPSS-split TSM (Driedger, Müller & Ewert 2014)	Detect transients and reset phase there; or split harmonic (stretch with PV) and percussive (stretch with OLA) and recombine	PV cost + HPSS (~1.5–2×)	Lowest among classic methods	Best choice for full-mix music
Phase-gradient heap integration (PGHI; Průša & Holighaus 2017)	Reconstructs phase from magnitude gradients; no phase-unwrapping errors	Similar to PV	Low; fewer phasiness artefacts at large ratios	Research-grade implementations
Rubber Band (R2 "faster", R3 "finer")	PV hybrid with transient detection, band-split phase handling, formant option	R2 a few % of a core; R3 roughly 2–4× R2 (estimate, **measure**)	Very low; R3 is the open-source reference	**GPL-2.0+, compatible with Riff's GPL-3.0.** Recommended
Commercial (zplane élastique, Serato Pitch'n Time, Superpowered)	Proprietary PV/OLA hybrids	Low to medium	Very low	Not GPL-compatible to bundle
Neural TSM (research, 2020–2025)	Neural vocoders or diffusion models re-synthesising at a new rate	Far beyond a phone in real time	Can hallucinate or blur transients; not yet better than R3 at small ratios	Not recommended; small-ratio stretching is a solved problem classically

Policy for Riff.

- Stretch the **incoming** song toward the outgoing one. Its intro is usually sparser, and listeners judge "the song I was hearing" more harshly.
- Ramp the rate in over 2 beats *before* it becomes audible and back to 1.0 over 8 bars after A is gone. A rate ramp is inaudible; a step is not.
- Prefer speeding up to slowing down when using Sonic or WSOLA; with Rubber Band both directions are fine.
- Cap at ±4 %; between 4 and 8 %, use a tempo blend (both decks meet halfway, each ≤ 4 %) or don't beat-match (3.5).

Failure modes	Stretcher latency shifts B by tens of ms: measure each stretcher's group delay once with a click track and subtract it in the frame mapping. Stretchers need pre-roll: start feeding B at least 1 s before it becomes audible. Rate changes within a stretcher are smooth only if the library supports time-varying ratio without reset (Rubber Band real-time mode does). Hand-over from the stretched B to the normal player at 1.0 must be sample-continuous: ramp back to exactly 1.0 before switching paths.

3.3 Key detection and tuning

Method	How	Cost	Failure modes
Template matching: Krumhansl–Kessler, Temperley, EDMA profiles (Faraldo et al. 2016, for electronic music)	Harmonic-weighted chroma averaged over the window; Pearson correlation with 24 rotated profiles	Microseconds after chroma	Relative major/minor confusion; modal music (Dorian, Mixolydian); tracks with no harmonic content (drum tools). EDMA profiles help on EDM, where minor keys dominate
CNN key classifier (Korzeniowski & Widmer, EUSIPCO 2017; ISMIR 2018 genre-agnostic)	Log-frequency spectrogram → 24 classes	Small model, < 1 MB; tens of ms per window (estimate)	Needs own training data for a shippable licence **[verify]**
Tuning offset (rarely discussed, important)	Circular mean of spectral peak positions modulo 100 cents: $\delta = (100/2\pi)\cdot\arg \Sigma_k \lvert X_k\rvert^2 \cdot e^{j2\pi\cdot c_k/100}$, with c_k each peak's pitch in cents from A440	Microseconds	Unreliable on drum-dominated windows: require harmonic energy share ≥ 0.3

Why tuning matters. Two songs in the "same" key but tuned 30 cents apart (old records, tape-speed masters, A = 432 Hz productions) beat against each other on sustained notes: at 440 Hz, 30 cents is 7.7 Hz of beating. If $|\delta_A - \delta_B| \geq 15$ cents and both keys are confident, pitch-shift B by $\delta_A - \delta_B$ cents during the blend. Rubber Band takes pitch scale and time ratio independently, so this combines with the tempo stretch at no extra cost.

Camelot mapping and the compatibility score table are in the 28 Oct report (2.5); they are used unchanged here as $K(A, B) \in [0, 1]$, scaled by key confidence.

3.4 Harmonic decision logic: transpose, shorten, or change the transition type

```
inputs: K = key compatibility (0…1, confidence-weighted), harmonicShare of each window (0…1),
        Δ = folded tempo ratio error, V = predicted vocal overlap, grid confidences
if harmonicShare_A < 0.25 or harmonicShare_B < 0.25:      # drum intro/outro: key is irrelevant
    key gates nothing; any length allowed
elif K ≥ 0.85:  long blend allowed (16–32 bars)
elif K ≥ 0.60:  blend ≤ 8 bars; prefer bass-swap EQ blend
elif K ≥ 0.45 and |semitones to nearest compatible key| == 1 and keyConf_A, keyConf_B ≥ 0.2:
    transpose B by ±1 semitone from the start of the blend, keep it there until B's next section
    boundary, and return to the original pitch on that boundary (a glide back is audible; a jump
on a
    boundary is not)
else:           blend ≤ 2 bars, or a filter-swap (4.3) / echo-out (4.5) / drums-only stem overlap
(4.4)
```

Cost	Decision: nothing. Transposition: the stretcher's pitch path (already running if beat-matching).
Failure modes	Transposing vocals by a semitone is audible (formant shift) on well-known songs; restrict to sections without vocals, or use formant preservation. Returning to the original pitch mid-song is audible unless done at a section boundary or during a breakdown. Key estimates of 40–60 % accuracy on some genres mean a "clash" verdict is often wrong; that is why low confidence relaxes the gate rather than tightening it.

4. Transition DSP and typologies

4.1 Gain laws: correlation decides, not taste

For two signals of equal level with correlation coefficient `ρ`, the power of the mix is $g_A^2 + g_B^2 + 2\rho \cdot g_A g_B$. Equal-power gains (`cos/sin`) are flat only for `ρ = 0`; for beat-aligned kicks or matched basslines (`ρ → 1`) they bulge by +3.01 dB at the midpoint. A correlation-aware family that is flat for any `ρ`:

```
a(x) = cos(πx/2),  b(x) = sin(πx/2),  N(x) = √(1 + 2ρ·a(x)·b(x))
g_A(x) = a(x)/N(x),   g_B(x) = b(x)/N(x)
ρ = 0  → equal power;   ρ = 1 → N = a + b, so g_A + g_B = 1 (constant amplitude, linear-sum)
```

Estimate `ρ` **per band, offline**: correlate the time-aligned, band-passed transition windows of A and B (ρ_{low}, ρ_{mid}, ρ_{high}). Unaligned songs give ρ ≈ 0 everywhere; beat-matched four-on-the-floor gives ρ_{low} of 0.3–0.7 during kicks.

Cost	One square root per control block.
Failure modes	ρ varies within the blend (kick on, kick off). Using one value per band for the whole blend is fine; per-beat ρ makes the gain jitter audibly. Negative ρ (polarity-inverted bass, rare) would need N < 1; clamp ρ to [0, 1].

4.2 Three-band EQ and the bass swap

Crossovers

- **Low/mid at 120 Hz, LR4** (two cascaded Butterworth biquads per side). Kick fundamentals are 45–100 Hz and sub-bass 30–80 Hz, so 120 Hz keeps the "thump" and sub on the low side while bass harmonics, which carry pitch, stay in the mid band and blend smoothly. The 28 Oct report gives coefficients at 48 kHz and shows the halves sum to 0.000 dB.
- **Mid/high at 2.5 kHz, LR4**: hats, cymbals and vocal sibilance above; vocal body and chords below.
- **Phase-correct three-band split:** split low/rest at 120 Hz, then rest into mid/high at 2.5 kHz, and pass the low band through a 2.5 kHz *all-pass* of matching order. Otherwise the three bands don't sum flat. That's 10 biquads per channel per deck.

DJ mixer EQs differ (shelves at roughly 70 Hz, 1 kHz and 13 kHz on common club mixers; "isolators" with crossovers near 200–300 Hz and 2–4 kHz) **[verify per model]**. For an automated engine the crossover form is better: a band at unity is bit-transparent in magnitude, and a band at 0 is a true kill, not −26 dB.

Choreography of an 8-bar EQ blend

The standard club move, as automation lanes (bar 1 = first bar of the blend, x = progress inside the stated bars):

Bars	Deck B (incoming)	Deck A (outgoing)
1–2	High band 0 → 1 (sin), mid at −12 dB, low 0 (killed)	Unchanged
3–4	Mid −12 dB → 0 dB (sin in amplitude)	High 1 → 0.71 (−3 dB) to make room for B's hats
5, on the downbeat	**Low 0 → 1** (hard swap, or 1 beat linear amplitude)	**Low 1 → 0** at the same instant, complementary
5–8	All bands at unity	Mid and high → 0 (cos); optional low-pass sweep 20 kHz → 400 Hz

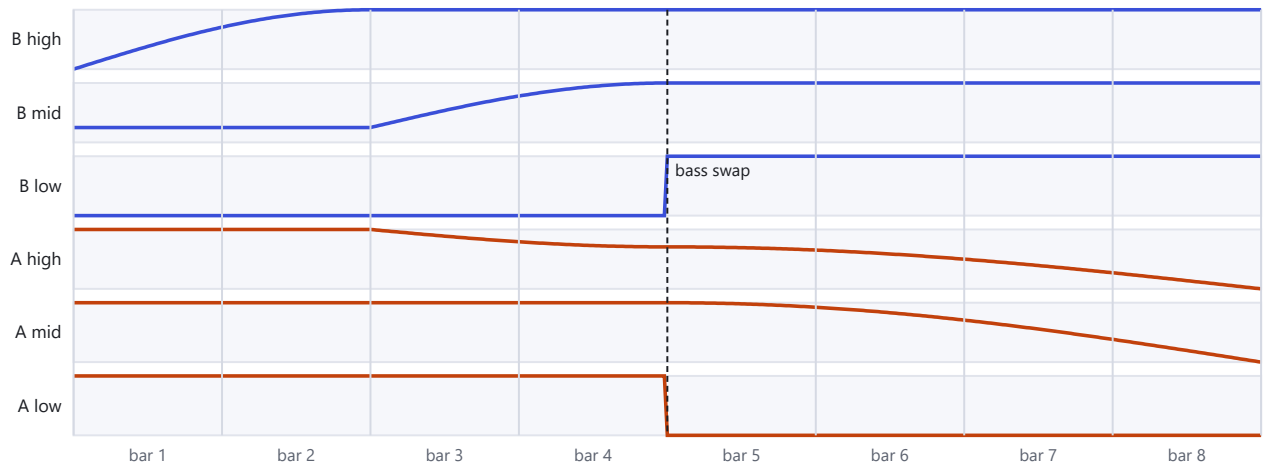


Figure 1. Band gains (linear amplitude, 0–1) for the 8-bar choreography. The low band is never shared: one deck carries it at unity until the swap downbeat, the other after.

Today's bass swap, measured against this

Riff 0.20.0 applies the equal-power `cos/sin` to each player's whole output, and on top, a 150 Hz low shelf moved linearly in dB: outgoing 0 → −26 dB and incoming −26 → 0 dB over 1 s centred on the blend midpoint. Two consequences, computed for an uncorrelated low band (the usual case for two songs not phase-aligned):

- **Bass hole at the swap:** at its centre both shelves are at −13 dB, so low-band power is $0.5 \cdot (10^{-1.3} + 10^{-1.3})$ = −13.0 dB relative to one song alone, about 10 dB under the level just before the swap.
- **Sag before and after:** the low band carried by one deck still follows that deck's full-band fade. A's bass falls to −3 dB by the midpoint and B's starts at −3 dB, so the low end goes 0 → −3 → (−13) → −3 → 0 dB.

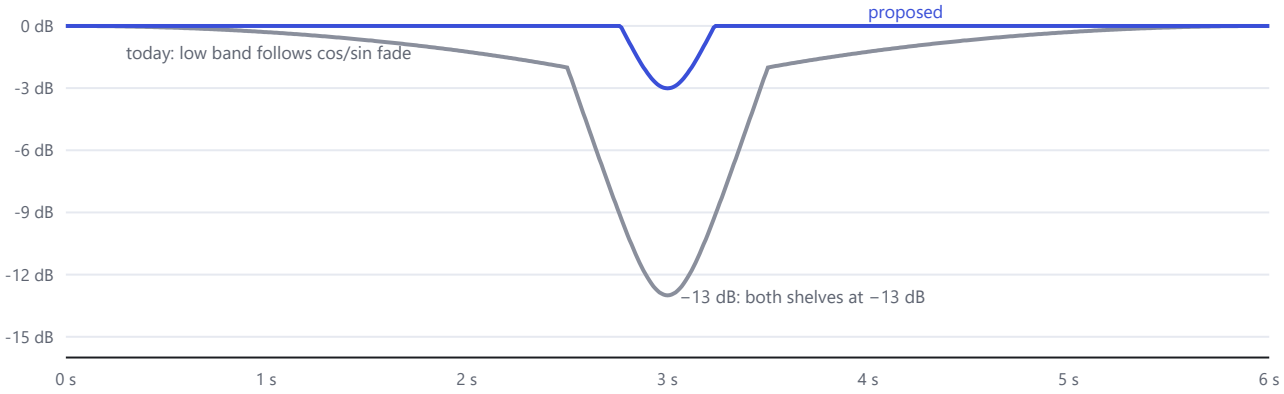


Figure 2. Low-band level (below the shelf corner, uncorrelated sum) through a 6 s blend with a 1 s swap at its midpoint. Grey: Riff 0.20.0 (equal-power full-band fade × linear-in-dB shelves). Blue: proposed (low band outside the full-band fade, amplitude-complementary swap over 1 beat at 128 BPM). The proposed curve dips −3 dB only within the 0.47 s swap, and 0 dB if the basses are phase-aligned ($\rho_{low} = 1$).

```
Proposed (Phase 1, works on the two-player design too):
  per deck output = g_full_midhigh · HighShelfPart + g_low · LowPart     (or the LR4 split once
the mixer exists)
  g_low_A = 1 − u,  g_low_B = u,   u = clamp((t − t_swap)/T_swap, 0, 1)   (amplitude, NOT dB)
  T_swap = 1 beat when the swap is on a confident downbeat; 1 s otherwise; 0 (hard) if both
downbeat conf ≥ 0.7
  With the shelf available today, set each cut so its linear gain follows u:
      incoming cut_dB = 20·log10(max(u, 10^(−26/20))),   outgoing cut_dB = 20·log10(max(1 − u,
10^(−26/20)))
  and divide the low band out of the player-volume fade (apply volume = 1 to the deck that carries
the bass,
  and move the mid/high fade to a high shelf or, better, to the mixer's band gains).
```

Cost	LR4 three-band split: 10 biquads × 2 channels × 2 decks ≈ 40 biquads at 48 kHz ≈ 10 M multiply-adds per second: under 1 % of one mid-range core (estimate).
Failure modes	Float32 biquads at 120 Hz / 48 kHz have poles near the unit circle ($a_2 \approx 0.978$): use TPT/SVF form (as Riff's equalizer already does) or double-precision state. Changing band gains in steps causes zipper noise: interpolate per 64-sample block. A hard swap on a mis-detected downbeat is a train wreck: require downbeat confidence. Clipping: two full-level decks during the mid-blend plus coherent kicks can peak +6 dB: keep 3 dB of headroom in the mix bus and let the true-peak limiter (6.3) catch the rest.

4.3 Filter sweeps and the filter swap

How	**Sweep-out:** low-pass A with an exponential cutoff $f(x) = f_{hi} \cdot (f_{lo}/f_{hi})^x$ (20 kHz → 300–400 Hz), resonance Q 0.9–1.4. **Rise-in:** high-pass B, 300 Hz → 20 Hz. **Filter swap for key clashes:** run both at once so A's spectrum retreats upward (HPF rising to 600 Hz) while B's opens from below (LPF opening from 600 Hz). The two songs' harmonic content overlaps in frequency only briefly, so the clash is heard as timbre movement, not wrong notes. Use a TPT state-variable filter (Zavalishin / Simper), which stays stable and click-free under fast cutoff modulation; Riff's equalizer already uses this structure.
Cost	One SVF per channel per deck: negligible. Recompute coefficients per 64-sample block.
Failure modes	Resonance adds gain: an SVF low-pass peaks at about Q at its cutoff (Q 2 = +6 dB). Sweeping through a bass-heavy region with Q > 1.4 can clip: scale input by 1/√Q or limit Q. A linear (not exponential) cutoff ramp spends almost all its time in the highs and then rushes through the audible range. Filtering removes energy, so perceived loudness drops; the predictive loudness lane (6.3) should treat filters as gain reductions.

4.4 Stem-based mixing

How	**Separation:** modern models split a mix into vocals, drums, bass and "other" (HTDemucs, the hybrid transformer of Rouard, Massa & Défossez 2023; band-split RNN and BS-RoFormer, 2023). djay's Neural Mix runs such a model in real time on Apple's Neural Engine; real-time causal variants exist in research (HS-TasNet, ICASSP 2024) **[verify]**. **Mixing:** each deck becomes four faders, so the engine can, for example: loop A's drum stem for 2 bars under B's melodic intro (a "drum bridge"); mute B's vocal stem until A's vocal ends (perfect clash avoidance); swap basses by stem instead of by EQ (no 120 Hz compromise); play A's acapella over B's instrumental for one phrase (a mashup moment, only when keys match, $K \geq 0.85$).
Cost	**Real time on mid-range Android: impractical.** HTDemucs-class models have tens of millions of parameters and run near real time on desktop CPUs; on a phone CPU expect several times slower than real time (estimate). **Key insight for Riff: AutoMix doesn't need real-time separation.** It needs the 30–45 s transition windows separated *before* the blend. Separating 75 s per song in the background (WorkManager, charging, unmetered), even at 5× slower than real time, is ~6 minutes per song: fine for downloads and the next items of a queue planned overnight, not for streamed songs chosen a minute ahead. Storage: 4 stems × 75 s as Opus 64 kb/s stereo ≈ 2.4 MB per song; or store soft masks (8-bit, 64 mel bands, 50 fps ≈ 1 MB) and re-apply them to the decoded window at play time.
Failure modes	Separation artefacts are masked in a full mix but exposed when a stem is soloed (a looped drum stem with vocal bleed is worse than no loop). Sum of stems ≠ original (some models aren't mixture-consistent): play the original whenever all four faders are at unity, and use stems only for the moves that need them. Loop seams: cut loops on beat-aligned zero crossings with 5 ms crossfades. Licences: check each model's *weights* licence separately from its code (the 28 Oct report found HTDemucs weights' licence disputed). Thermal: a long separation job on a phone throttles it; run only when charging and cool (thermal headroom API).

Cheap "pseudo-stems" for Phase 2, with no model: drums ≈ the percussive part of HPSS; bass ≈ LR4 low band; vocals ≈ the harmonic part's mid channel (L + R) in 300–3,400 Hz, where lead vocals are usually panned. Crude, but enough to duck B's vocal region or emphasise A's drums during a bridge.

4.5 Creative exit effects for tempo and genre jumps

Effect	How it works (parameters)	Cost	Failure modes
Echo-out	Feedback delay on A, time = 1 beat (or ¾ beat) at A's tempo; feedback 0.5–0.6 (0.55 decays −5.2 dB per repeat: 60 dB in 11.6 repeats ≈ 5.4 s at 128 BPM); HPF 200 Hz and gentle LPF in the loop so repeats thin out; send 0 → 1 over A's last bar; A's dry cut on the downbeat; B starts on its own "1" one or two beats later	One delay line (≈ 2 s × 2 ch)	Echoing a vocal word mid-syllable sounds like a glitch: cut at a vocal gap. Echo tails at A's tempo over a much slower B flam against B's beat: keep the send's feedback lower (0.4) when Δ > 15 %
Delay freeze	Capture A's last beat or half-beat into the delay with feedback 1.0 and input cut: an infinite stutter; LPF it from 8 kHz to 300 Hz over 1–2 bars while B rises in	Same	Feedback exactly 1.0 with loop filters below unity decays anyway; with gain > 1 it blows up: hard-clamp loop gain ≤ 1
Reverb wash	Send A into a long reverb (RT60 3–6 s, pre-delay 1/16 note), cut A's dry on the downbeat, B starts under the tail; duck the reverb's lows below 200 Hz so B's kick stays clean	A Schroeder/FDN reverb: 4–8 delay lines, ~1–2 % of a core (estimate)	Washy and "cheap" if overused; keep for genre jumps, not as a default
Vinyl brake / tape stop	Resample A with rate $r(t) = (1 - t/T)^p$; content consumed is $T/(p+1)$ (T = 1 s, p = 2 → 0.33 s of music). End the brake on A's next downbeat, B's "1" there	Variable-rate resampler	Pitch drops a lot (intended); without a low-pass during the brake, aliasing as rate approaches 0: interpolate with a windowed sinc whose cutoff follows the rate

Effect	How it works (parameters)	Cost	Failure modes
Loop roll / beat-repeat	Loop A's last 1 beat, then ½, ¼, ⅛ (a "roll") over one bar, then cut; B on the next downbeat	Tiny	Needs a confident beat grid; on swung music the halving sounds off
Silence drop	Cut A dead on a phrase-end downbeat; one beat of silence; B starts on its strongest "1" (first drop or chorus)	Nothing	Only works when B's entry is strong; at a quiet intro it reads as a gap, not a drop

Worked example: 128 BPM house → 90 BPM hip-hop. The ratio is 0.703. Folding: ×1.5 gives 135 vs 128 (+5.5 %, beyond the 4 % stretch cap); ×1 is 30 % off; ×2 is 41 % off. So no beat-match. Plan: echo-out (or a 1-beat brake, 469 ms) on A's last confident phrase downbeat, ending exactly where A's next "1" would fall; B's first downbeat placed there (or one beat later after a silence drop). The listener hears a deliberate break, not a failed blend. Put B's entry at its first strong downbeat (verse or hook), skipping a long quiet intro if it has no vocals.

4.6 Transition selector

Condition (in order)	Transition	Length
Not eligible (album in order, gapless pair, podcast, classical, speed ≠ 1×, casting)	Riff's existing behaviour	—
No grid on either song (conf < 0.1), or analysis missing	Smart crossfade (stage 1) with proposed bass handling	3–12 s
Grids confident, folded Δ ≤ 4 %, same meter, phrase grid on both	**Beat-matched EQ blend** (4.2), B stretched	By K: 16–32 / 8 / ≤ 4 bars
Same, but K < 0.6 and harmonic windows	Beat-matched **filter swap** or drums-only overlap	2–4 bars
4 % < Δ ≤ 8 %	Tempo blend (each deck ≤ 4 %), or drop B on A's phrase downbeat without stretch	4–8 bars / 1–2 bars
Ratio near 3:2 or 2:1 within 4 %	Half/double-time alignment; bars counted at the slower tempo	4–8 bars
8 % < Δ ≤ 25 %	Echo-out or filter-out on A's phrase line; B on its own "1"	1–2 bars of overlap
Δ > 25 % or meter differs	Brake, freeze, silence drop or reverb wash	No rhythmic overlap
Predicted vocal overlap V > 1 beat after moving cues	Duck B's vocal band, or mute B's vocal stem	—

5. Industry benchmarks and academic state of the art

5.1 Products: what is known and what can be inferred

Product	Publicly known	Inferred mechanism (from behaviour and constraints)	Lesson for Riff
Apple Music AutoMix (iOS 26, 2025; Crossfade since iOS 17)	Analyses tempo and key; uses time-stretching and beat-matching; sometimes overlaps beats, sometimes fades or pauses; off for albums in order	Offline per-track analysis on Apple's servers or device, a planner choosing between beat-matched blends and fades by confidence, small stretch ratios; conservative about long overlaps **[inferred]**	Graceful degradation is the product: choose the safe transition when unsure. Riff already mirrors "off for albums"
Spotify AI DJ (2023) and **Smart Shuffle** (2023)	AI DJ: personalised sequence with a synthetic-voice host between songs; Smart Shuffle inserts recommendations into a playlist	Neither is primarily a transition DSP system: AI DJ's craft is in *sequencing* and the voice-over covers the seam; transitions are crossfades **[inferred]**	Sequencing (which song next) matters as much as the blend; a voice or sound bridge can hide an incompatible pair
Spotify Mix (Aug 2025, Premium)	Automatic or user-edited transitions on own playlists: presets (fade, rise, blend), volume and EQ curves, effects, BPM and Camelot key shown; applied in real time without altering tracks	Precomputed per-track analysis (Spotify has had beat/section analysis since the Echo Nest), a parametric lane model like the one in the 28 Oct report, executed by the client **[inferred]**	Exposing the plan as editable lanes is cheap once lanes exist. A Riff "edit this transition" UI is a natural Phase 3 item
Algoriddim djay Automix AI	Tempo modes (Automatic, Sync, Sync + Tempo Blend); transitions Fade, Filter, EQ, Echo, Dissolve, Neural Mix; ML-chosen intro/outro regions; real-time 4-stem Neural Mix on recent devices	Per-track ML cue detection (rhythmic intro/outro patterns), a library of parametric transitions, on-device separation on Neural Engines and recent Android NPUs **[inferred]**	The transition *vocabulary* (section 4) is the differentiator; stems are the premium tier
Pacemaker	AI DJ app that built continuous mixes from streaming catalogues; lost Spotify integration when Spotify ended third-party DJ access (2020), later moved to other catalogues **[verify current status]**	Server-side planning of cue points and transition types, client-side execution with EQ, filters and tempo; an "editor" exposing the AI's choices	Depending on a streaming partner's audio access is the main business risk; Riff's on-device PCM avoids it
Mixxx AutoDJ (open source, GPL)	Modes Full Intro + Outro, Fade at Outro Start, Full Track, Skip Silence; uses analyser-set intro/outro cues; Rubber Band key-lock	Readable source: the closest GPL reference for cue handling and Rubber Band integration	Riff can study (and, licence permitting, adapt) its handling of intro/outro cues and edge cases

5.2 Academic state of the art, by topic

Topic	Key work (details in section 9)	What it contributes to AutoMix
End-to-end automatic DJ systems	Cliff 2000 (HP Labs, "Hang the DJ"); Ishizaki et al. ISMIR 2009 (tempo adjustment minimising discomfort); Vande Veire & De Bie 2018 (drum & bass auto-DJ, EURASIP)	Pipeline templates; Vande Veire's system is the most complete published design with structure-aware cue selection for one genre

Topic	Key work (details in section 9)	What it contributes to AutoMix
Analysis of real DJ mixes	Kim, Choi, Sacks, Yang & Nam, ISMIR 2020 (mix-to-track alignment over 1,500+ mixes); Kim, Yang & Nam, NIME 2021 (reverse-engineering EQ/fade curves); Schwarz & Fourer (DJ-mix reverse engineering datasets)	Ground truth for how humans mix: typical overlap lengths, which bands move when, cue positions relative to phrases. Use to set Riff's defaults and to train a classifier
Cue points	Zehren, Alunno & Bientinesi, CMJ 2022 and M-DJCUE dataset; cue points as object detection (arXiv 2024)	Supervised targets for cue scoring (2.6)
Learned transitions	Chen et al., ICASSP 2022 (DJtransGAN: differentiable EQ and fader effects trained adversarially on real mixes)	Learns *parameter curves*, not audio: compatible with a lane executor; a Phase 3 direction
Mashups and compatibility	Davies et al., AutoMashUpper (ISMIR 2013, TASLP 2014)	Harmonic and rhythmic "mashability" measure: a better K(A, B) than Camelot distance
Sequencing and continuation	Bittner et al. (Spotify), ISMIR 2017 "Automatic playlist sequencing and transitions"; RecSys Challenge 2018 / Million Playlist Dataset (Chen, Lamere, Schedl & Zamani)	Reordering a queue for smooth transitions is often worth more than a better blend. Riff could reorder within shuffle (Phase 3)
Beats and downbeats	Böck, Krebs & Widmer ISMIR 2016 (joint RNN + DBN); Böck & Davies ISMIR 2020; Foscarin, Schlüter & Widmer ISMIR 2024 (Beat This!, in Riff)	Riff already ships the current state of the art
Structure	Foote 2000; McFee & Ellis ISMIR 2014; Serrà et al. 2014; Ullrich et al. ISMIR 2014; Kim & Nam WASPAA 2023 (allin1)	Section 2
Singing voice detection	Schlüter & Grill ISMIR 2015; Lee, Choi & Nam ISMIR 2018	Section 2.7
Key	Krumhansl 1990; Temperley 1999; Faraldo et al. 2016 (EDM); Korzeniowski & Widmer 2017/2018	Section 3.3
Time-scale modification	Laroche & Dolson 1999; Röbel 2003; Driedger & Müller 2016 review; Průša & Holighaus 2017	Section 3.2
Source separation	Défossez 2021 (Hybrid Demucs); Rouard et al. ICASSP 2023 (HTDemucs); Luo & Yu 2023 (BSRNN); Lu et al. 2023 (BS-RoFormer)	Section 4.4
Loudness	ITU-R BS.1770; EBU R128 and Tech 3341/3342	Section 6.3

6. System architecture and performance

6.1 Two-stage pipeline

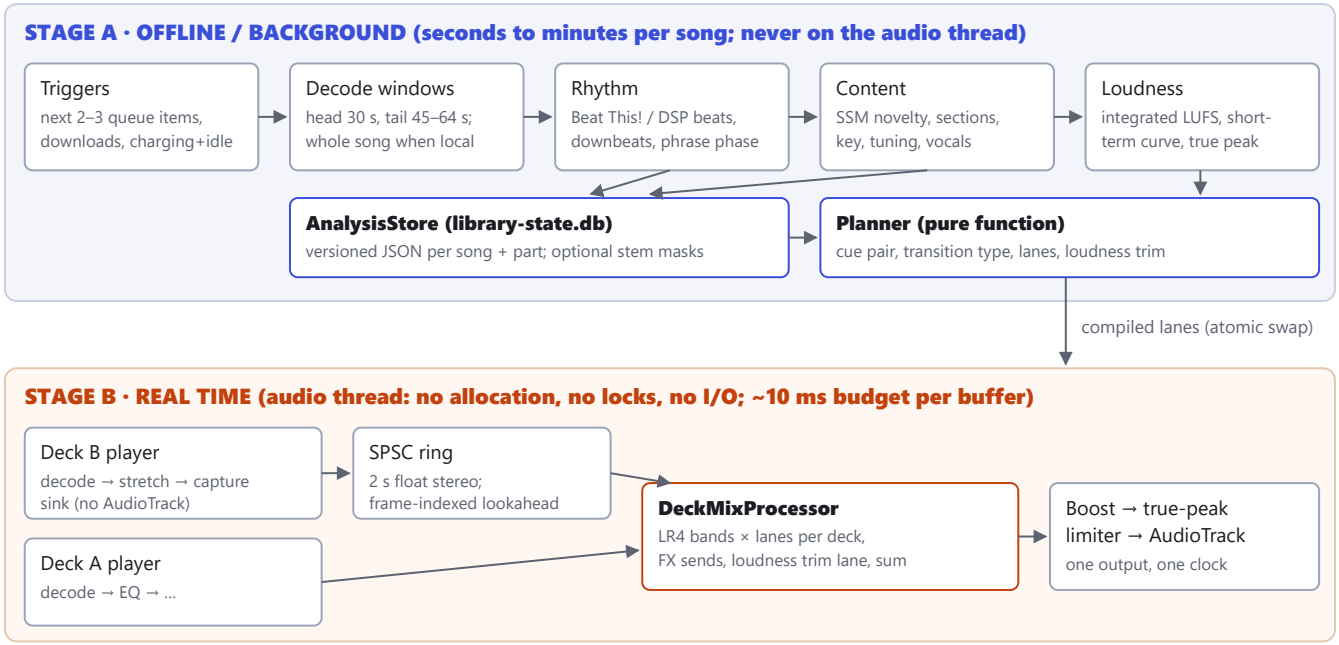


Figure 3. Stage A analyses ahead of time and plans; stage B only executes compiled lanes on one output. Today Riff has stage A's first half (windows, beats, envelopes, AnalysisStore) and runs stage B as two players.

Stage A: what to compute and keep

Item	Resolution	Phone cost per song (estimate unless stated)
Beats, downbeats (Beat This!)	Times in ms; confidences	**Measured:** 3.2–7.3 s per window, load 2.0–3.6 s
Phrase phase per P ∈ {4, 8, 16}, with confidence	Per window	< 1 ms
Beat-synchronous features, novelty, section labels	Per beat	5–20 ms
Key, key confidence, tuning offset (cents), harmonic share	Per window	< 5 ms after chroma
Vocal curve (lyrics prior + detector)	10 Hz	10–50 ms
Loudness: integrated (song), short-term 3 s and momentary 400 ms curves (windows), true peak	10 Hz curves	10–30 ms
Cue candidates with scores	List	< 1 ms
Stem masks (Phase 3, optional)	8-bit, 64 bands, 50 fps	Minutes, while charging only

Store as a versioned JSON payload per `songId:part` (Riff's `AnalysisStore` already keys by part and version; the model's analyses are version 2). Bump the version per analyser change so old results are recomputed lazily, never all at once.

Scheduling. Today analysis is triggered by the silence watch during playback. Add a WorkManager job: analyse the next 2–3 queue items as soon as they're known (local and cached ones), and the download library when charging, idle and cool (`PowerManager.getThermalHeadroom` ≥ 0.3 of headroom, or status < MODERATE). Keep ONNX at one or two intra-op threads at background priority (an open owner question in the 0.20.0 handover).

6.2 Stage B: the real-time path

- **One output, one clock.** Deck B's player renders into a capture sink that writes to a lock-free single-producer/single-consumer ring; deck A's processor chain pulls exactly the frames it needs and sums them. Alignment becomes an index calculation, with no feedback loop against two hardware clocks. Design details are in the 28 Oct report (3.1).
- **Lookahead buffer:** 2 s of float stereo (192 k frames) is enough: the stretcher's latency (≤ 100 ms) plus jitter in B's decode. B is armed 20 s ahead for streams, so the network has time.
- **Control vs audio thread:** the planner's lanes are compiled to output-frame times on the control thread and swapped in with an atomic reference. The audio thread evaluates lanes per 64-frame sub-block (1.33 ms at 48 kHz): fine enough for any fade, filter or bass swap.
- **Budget:** mixer and filters < 1 % of a core; Rubber Band R3 the main cost (measure); everything else is stage A.

Failure modes	Underrun of B's ring (slow decode, thermal throttling): mix silence for B and abort to a plain fade at the next control tick, never stall A. Media3 can swap a sink only by rebuilding the renderer: do the role swap at the song boundary with a 20 ms bridge. Any user action (seek, skip, pause > 2 s, queue edit) aborts: restore A's lanes to unity over 50 ms.

6.3 Loudness: normalise, predict, then limit

1. **Normalise each song** to a target integrated loudness (BS.1770 K-weighting: high-shelf pre-filter around +4 dB above ~1.5 kHz and a ~38 Hz high-pass; 400 ms blocks with 75 % overlap; absolute gate −70 LUFS, relative gate −10 LU). −14 LUFS matches streaming practice; ReplayGain 2.0 uses −18 LUFS. Riff already normalises per track; keep its target and add a true-peak check.
2. **Predict the blend's loudness offline.** Both windows' short-term loudness curves $L_A(t)$, $L_B(t)$ are known, and so are the planned gains. For uncorrelated content:

   ```
   L_mix(t)   = 10·log10( Σ_bands  g_A,b(t)²·10^(L_A,b(t)/10) + g_B,b(t)²·10^(L_B,b(t)/10) )
   L_target(t)= linear in time from L_A(blend start) to L_B(blend end)      (bridges level differences)
   trim(t)    = clamp( L_target(t) − L_mix(t), −3 dB, +1.5 dB ), smoothed zero-phase over 1 s (offline, so no lag)
   ```

   Add it as one more lane. Because it is computed before the blend, it has no attack or release and doesn't pump.
3. **Don't "fix" musical dynamics.** An intro that is 8 dB below its song's body is intended; the target is the transition's continuity, not a flat line. Limit positive trim to +1.5 dB.
4. **Final safety:** a true-peak limiter (4× oversampled peak detection per BS.1770 annex, 1.5–5 ms lookahead, ceiling −1 dBTP) after the mix and boost. Riff's existing limiter in the boost stage can take this role if its detector is made true-peak.

Cost	Prediction: microseconds at plan time. True-peak detection: a 4× polyphase upsampler (≈ 48 taps) per channel ≈ 1 % of a core (estimate).
Failure modes	Coherent content (beat-matched kicks) adds up to 3 dB more than the uncorrelated prediction: include ρ per band (4.1). Short-term loudness lags transients by its window: use momentary (400 ms) curves for blends shorter than 4 s. A song whose stored integrated loudness was measured on a different edit (YouTube vs local file) will be off: key loudness to the analysed file, not the song ID.

> **The 31 Oct "+6 dB at handover" bug** (two copies playing for a moment as the tail player's hand-over landed on its own thread) is exactly what a single mixer rules out by construction. Until then, keep the regression test that both players' summed gain never exceeds the equal-power envelope by more than 0.5 dB.

6.4 Libraries and tools

Library	Language	Use in AutoMix	Licence vs Riff (GPL-3.0)	On the phone?
Essentia	C++ (Python)	Everything: beats, key (incl. EDMA profiles), loudness (EBU R128), HPCP, segmentation	AGPL-3.0 code: combinable; models CC BY-NC-ND: no	Possible via NDK, heavy build; better as offline reference
librosa	Python	Prototyping: onsets, beat_track, HPSS, chroma, recurrence matrices, Laplacian segmentation	ISC	No (reference only)
madmom	Python/Cython	RNN+DBN beats/downbeats, key; evaluation baseline	Code BSD; models CC BY-NC-SA: no shipping	No
aubio	C	Light real-time onsets, tempo, pitch	GPL-3.0: compatible	Yes, small
Beat This!	PyTorch / ONNX	Beats and downbeats (in Riff)	MIT (per secondary sources) **[verify]**	Yes (shipped)
Rubber Band	C++	Time-stretch and pitch-shift (R2/R3), real-time mode	GPL-2.0+: compatible	Yes, NDK, ~0.5–1 MB
SoundTouch	C++	WSOLA stretch	LGPL-2.1	Yes
Sonic	C / Java	Speech-tuned stretch (Media3's)	Apache-2.0	Already present
Pedalboard (Spotify)	Python (JUCE inside)	Offline rendering of test transitions: filters, reverb, delay, Rubber Band stretch	GPL-3.0	No: the evaluation harness
Spleeter	Python / TensorFlow	2/4/5-stem separation; quality now below Demucs	MIT	Offline only
Demucs (HTDemucs)	PyTorch	Best-quality 4-stem separation; evaluation and stem prototyping	Code MIT; weights disputed **[verify]**	Background only, if at all
Open-Unmix	PyTorch	Lighter separation	Code MIT; weights per README **[verify]**	Background, int8
ONNX Runtime / LiteRT (TFLite)	C++ / Java	Model execution; NNAPI or GPU delegates	MIT / Apache-2.0	Yes (ORT shipped)
PFFFT / KissFFT	C	Fast FFT with NEON (PFFFT) for a native port	BSD-style	Yes
Mixxx	C++	Reference AutoDJ, intro/outro cues, Rubber Band integration	GPL-2.0+	Study only

7. Prioritised implementation roadmap

Ordered by impact per unit of effort and risk, mapped to Riff's stages. Each item lists how to know it worked.

7.1 Phase 1: low-hanging fruit (days each; Kotlin only, no new dependencies)

#	Change	Why, cost, risk	Done when
1.1	**P1** Fix the bass hole and sag (4.2)	Move the low band's cut to follow `u` in linear amplitude, and take the deck carrying the bass out of the full-band fade (or re-express the mid/high fade as a high-shelf/volume pair). Biggest audible improvement per line of code. Risk: low; easy to A/B.	Rendered blend's < 120 Hz level stays within −3 dB through the swap (offline test on a fixture pair)
1.2	**P1** Phrase-anchored start (2.3)	Replace `index % 4` with a phrase-phase estimate, falling back to counting back from the outro start. Cost: tiny. Risk: low.	On the 50-song set, blend starts land on human-marked phrase lines ≥ 70 % (vs ~25 % by chance)
1.3	**P1** Swap on the downbeat	Centre the bass swap on the confident downbeat nearest the midpoint, 1 beat long, instead of a fixed 1 s at the exact midpoint. Cost: tiny.	Swap time within 10 ms of a tracked downbeat whenever conf ≥ 0.5
1.4	**P1** Lyrics as a vocal prior (2.7)	Use synced lyric line times to reject cue pairs with vocal overlap > 1 beat. Cost: small (data already present). Risk: lyric offsets; cross-check against the audible start.	No vocal-over-vocal blends in the 30-pair listening set where lyrics exist
1.5	**P1** Echo-out for incompatible tempos	When the folded tempo error is > 8 % and A's grid is confident: 1-beat feedback delay on A's last phrase downbeat (4.5), B on its own downbeat. One delay line in the existing processor chain. Risk: needs the tail player to carry an extra processor.	Listening score for "genre jump" pairs ≥ current crossfade + 0.5
1.6	**P1** Predictive loudness trim (6.3)	Short-term loudness of both windows is a by-product of the existing 100 ms envelope; compute the trim curve at plan time and apply it as a gain multiplier. Cost: small.	Short-term loudness during the blend within ±1.5 LU of the pre-blend level on 95 % of pairs
1.7	**P1** Longer tail window for local songs	45 s → 64 s for local files and downloads (32 bars at 120 BPM). +40 % decode and model time on those songs only. Risk: the model's cost; gate by the owner's open question on the model's load.	32-bar phrase confidence available on ≥ 50 % of 4/4 songs

7.2 Phase 2: intermediate (weeks; one native dependency)

#	Change	Why, cost, risk	Done when
2.1	**P2** Single-output mixer (capture sink + `DeckMixProcessor`)	Prerequisite for 2.2–2.5 and most of Phase 3. Large: Media3 sink plumbing, the handover, abort paths. Risk: glitches at the handover; mitigate with JVM tests on a fake sink and the 20 ms bridge.	Click-track alignment error ≤ 2 ms on the phone; no level jump > 0.5 dB at the handover
2.2	**P2** LR4 three-band lanes and the 8-bar choreography (4.2)	Replaces the shelf; true kills; correlation-aware gains per band. Small once 2.1 exists.	Bands sum flat (±0.05 dB) at unity; listening ≥ stage 2 + 0.5

#	Change	Why, cost, risk	Done when
2.3	**P2** Beat-matching with Rubber Band (stage 3)	B stretched ≤ ±4 % with phase-function alignment and a gentle correction loop (3.1). NDK build ~0.5–1 MB. Gate: stage 2/4 beat accuracy targets on the 50-song set.	Phase error ≤ 5 ms over a 16-bar blend on steady-beat pairs; no audible stretch artefacts in blind tests
2.4	**P2** Structure and cue scoring (2.2, 2.6)	Beat-synchronous novelty, section labels, joint cue-pair search. Spectral clustering for whole songs in the background.	Chosen cue pairs agree with the owner's own picks ≥ 60 % on 30 pairs
2.5	**P2** Key, tuning and harmonic gating (3.3, 3.4)	Chroma (already near the STFT), profiles, tuning offset, transition-type selector (4.6), filter swap.	MIREX-weighted key score ≥ 0.65; no "wrong-note" ratings in clash pairs
2.6	**P2** Background analysis with WorkManager (6.1)	Analyse queued and downloaded songs while charging and cool. Moves the model's cost off active playback, which also answers the owner's 31 Oct question about its load.	≥ 90 % of transitions in a playlist session have both analyses before the blend
2.7	**P2** Vocal detector (HPSS heuristic) and B-vocal ducking	For songs without synced lyrics.	Frame accuracy ≥ 0.8 against separated vocals on the test set

7.3 Phase 3: cutting-edge (months; research risk)

#	Change	Why, cost, risk	Done when
3.1	**P3** Pre-separated stems for transition windows (4.4)	Separate the 75 s windows of downloaded and queued songs while charging; store masks; enable drum bridges, vocal-mute overlaps and stem bass swaps. Heavy compute (background), model licence checks, thermal limits.	Stem transitions rated ≥ EQ transitions + 0.5 on pairs with vocal clashes
3.2	**P3** ML transition classifier	Learn the selector (4.6) and lane parameters from (a) real DJ mixes aligned to their tracks (the ISMIR 2020 method) and (b) Riff's own signals, on the phone only: a skip within 10 s after a blend is a negative, a completed song a weak positive. A gradient-boosted tree or a small MLP over pair features (Δtempo, K, energies, section types, vocal overlap, genre). Privacy: training data never leaves the phone.	Offline: beats the hand-written selector on held-out DJ-mix transitions; online: lower post-blend skip rate
3.3	**P3** Learned parameter curves (DJtransGAN-style)	Predict EQ/fader curves with differentiable effects, executed by the same lanes. Training is off-device; inference is tiny.	Preferred over 2.2's fixed choreography in blind tests
3.4	**P3** Queue reordering for compatibility	Within shuffle, choose the next song among a few candidates by transition quality J (2.6) plus a novelty constraint (Bittner et al. 2017). Cheap once analyses exist. Risk: users notice "less random" shuffle; keep it opt-in.	Average J of consecutive pairs up ≥ 20 % without repeated artists closer than before
3.5	**P3** Generative transition bridges	Text- or audio-conditioned music models (MusicGen/Stable Audio-class) infilling 2–4 bars between A's tail and B's head at a shared tempo and key. Today far beyond a phone (billions of FLOPs per second of audio), and a server would send listening data off the phone, against Riff's privacy policy. Research watch only.	—
3.6	**P3** Editable transitions UI	Show the planned lanes as Spotify Mix does; let the user move cues or choose a type; store per pair. Requires stable lanes (2.2).	Owner's call; ask before designing

> **Decisions this report leaves to the owner** (per the project's rules): whether the analysis window may grow to 64 s (1.7) given the model's open load question; whether a native dependency (Rubber Band, 2.3) is acceptable in the APK; whether background analysis may run on battery or only while charging (2.6); whether shuffle may reorder (3.4); whether any model for separation or vocals may ship, and from which licence (3.1, 2.7).

8. Evaluation protocol

8.1 Objective measures (automated, on rendered blends)

Measure	Definition	Target
Beat alignment error	Cross-correlate onset envelopes of A and B in the blend; median absolute lag	≤ 5 ms (beat-matched), n/a otherwise
Phrase hit rate	Blend start within 1 beat of a human-marked phrase line	≥ 70 %
Loudness continuity	Max |short-term loudness − linear bridge| during the blend	≤ 1.5 LU
Low-end continuity	< 120 Hz level, as in figure 2	≥ −3 dB vs pre-blend throughout
Low-band clash	Time both decks' < 120 Hz bands are within 10 dB of each other	≤ 1 beat
Vocal overlap V	Section 2.7, with separated vocals as truth	≤ 1 beat on ≥ 95 % of blends
True peak	Max true peak of the output	≤ −1 dBTP
CPU and heat	Mixer core share during a blend; analysis CPU per song; thermal status in a 1-hour session	< 5 % mixer; no throttling from analysis

8.2 Listening test

30 pairs from the owner's playlists spanning: same-genre steady beat, tempo jumps, key clashes, vocal-heavy, live/rubato, genre jumps. Render each pair as current 0.20.0 Smart, current Automix (beta), and each new phase's version. Rate blind, randomised, 1–5 (or MUSHRA-style with the current blend as a hidden reference). Ship a change when its mean beats the current one by ≥ 0.5 and none of its renders scores 1, the same rule as the 28 Oct report.

8.3 In-use signals (on the phone only)

Skip within 10 s after a blend; manual seek during a blend; the user switching mode back to Crossfade. Kept in Riff's local event log only, as today, and used to tune weights (2.6) and train the classifier (7.3).

9. References

Compiled from the research literature; not re-fetched in this session. Check each before citing it outside the project. Product and API sources are listed in the 28 Oct engine report, section 7.

Automatic DJ systems and DJ-mix analysis

1. D. Cliff. "Hang the DJ: Automatic sequencing and seamless mixing of dance-music tracks." HP Laboratories Technical Report HPL-2000-104, 2000.
2. H. Ishizaki, K. Hoashi, Y. Takishima. "Full-automatic DJ mixing system with optimal tempo adjustment based on measurement function of user discomfort." ISMIR 2009.
3. L. Vande Veire, T. De Bie. "From raw audio to a seamless mix: creating an automated DJ system for Drum and Bass." EURASIP Journal on Audio, Speech, and Music Processing, 2018.
4. T. Kim, M. Choi, E. Sacks, Y.-H. Yang, J. Nam. "A computational analysis of real-world DJ mixes using mix-to-track subsequence alignment." ISMIR 2020.
5. T. Kim, Y.-H. Yang, J. Nam. "Reverse-engineering the transition regions of real-world DJ mixes using sub-band analysis with convex optimization." NIME 2021.
6. D. Schwarz, D. Fourer. "Methods and datasets for DJ-mix reverse engineering." CMMR 2019 / LNCS 2021. **[verify]**
7. M. Zehren, M. Alunno, P. Bientinesi. "Automatic detection of cue points for the emulation of DJ mixing." Computer Music Journal 46(3), 2022; M-DJCUE dataset.
8. Cue-point estimation as object detection. arXiv:2407.06823, 2024.
9. B.-Y. Chen, W.-H. Hsu, W.-H. Liao, M. A. Martínez Ramírez, Y. Mitsufuji, Y.-H. Yang. "Automatic DJ transitions with differentiable audio effects and generative adversarial networks." ICASSP 2022.
10. M. E. P. Davies, P. Hamel, K. Yoshii, M. Goto. "AutoMashUpper: Automatic creation of multi-song music mashups." ISMIR 2013; IEEE/ACM TASLP 22(12), 2014.

Sequencing and playlist continuation

11. R. M. Bittner, M. Gu, G. Hernandez, E. J. Humphrey, T. Jehan, P. H. McCurry, N. Montecchio. "Automatic playlist sequencing and transitions." ISMIR 2017.
12. C.-W. Chen, P. Lamere, M. Schedl, H. Zamani. "RecSys Challenge 2018: Automatic music playlist continuation." RecSys 2018.
13. A. Flexer, D. Schnitzer, M. Gasser, G. Widmer. "Playlist generation using start and end songs." ISMIR 2008.

Beats, structure, vocals, key

14. D. P. W. Ellis. "Beat tracking by dynamic programming." Journal of New Music Research 36(1), 2007.
15. S. Böck, F. Krebs, G. Widmer. "Joint beat and downbeat tracking with recurrent neural networks." ISMIR 2016.
16. S. Böck, M. E. P. Davies. "Deconstruct, analyse, reconstruct: How to improve tempo, beat, and downbeat estimation." ISMIR 2020.
17. F. Foscarin, J. Schlüter, G. Widmer. "Beat this! Accurate beat tracking without DBN postprocessing." ISMIR 2024.
18. J. Foote. "Automatic audio segmentation using a measure of audio novelty." IEEE ICME 2000.
19. B. McFee, D. P. W. Ellis. "Analyzing song structure with spectral clustering." ISMIR 2014.
20. J. Serrà, M. Müller, P. Grosche, J. L. Arcos. "Unsupervised music structure annotation by time series structure features and segment similarity." IEEE Transactions on Multimedia 16(5), 2014.
21. K. Ullrich, J. Schlüter, T. Grill. "Boundary detection in music structure analysis using convolutional neural networks." ISMIR 2014.
22. T. Kim, J. Nam. "All-in-one metrical and functional structure analysis with neighborhood attentions on demixed audio." IEEE WASPAA 2023.
23. D. Fitzgerald. "Harmonic/percussive separation using median filtering." DAFx 2010.
24. J. Schlüter, T. Grill. "Exploring data augmentation for improved singing voice detection with neural networks." ISMIR 2015.
25. K. Lee, K. Choi, J. Nam. "Revisiting singing voice detection: A quantitative review and the future outlook." ISMIR 2018.
26. C. L. Krumhansl. Cognitive Foundations of Musical Pitch. Oxford University Press, 1990 (Krumhansl–Kessler profiles, 1982).
27. Á. Faraldo, E. Gómez, S. Jordà, P. Herrera. "Key estimation in electronic dance music." ECIR 2016.
28. F. Korzeniowski, G. Widmer. "End-to-end musical key estimation using a convolutional neural network." EUSIPCO 2017; "Genre-agnostic key classification with convolutional neural networks." ISMIR 2018.

Time-scale modification, separation, loudness, DSP

29. W. Verhelst, M. Roelands. "An overlap-add technique based on waveform similarity (WSOLA) for high quality time-scale modification of speech." ICASSP 1993.
30. J. Laroche, M. Dolson. "Improved phase vocoder time-scale modification of audio." IEEE Transactions on Speech and Audio Processing 7(3), 1999.
31. A. Röbel. "A new approach to transient processing in the phase vocoder." DAFx 2003.
32. J. Driedger, M. Müller, S. Ewert. "Improving time-scale modification of music signals using harmonic-percussive separation." IEEE Signal Processing Letters 21(1), 2014.
33. J. Driedger, M. Müller. "A review of time-scale modification of music signals." Applied Sciences 6(2), 2016.
34. Z. Průša, N. Holighaus. "Phase vocoder done right." EUSIPCO 2017.
35. A. Défossez. "Hybrid spectrogram and waveform source separation." ISMIR 2021 MDX workshop.
36. S. Rouard, F. Massa, A. Défossez. "Hybrid transformers for music source separation." ICASSP 2023.
37. Y. Luo, J. Yu. "Music source separation with band-split RNN." IEEE/ACM TASLP, 2023.
38. W.-T. Lu, J.-C. Wang, Q. Kong, Y.-N. Hung. "Music source separation with band-split RoPE transformer." ICASSP 2024 (SDX 2023). **[verify]**
39. S. Venkatesh et al. "Real-time low-latency music source separation using Hybrid Spectrogram-TasNet." ICASSP 2024. **[verify]**
40. ITU-R BS.1770 (current revision): Algorithms to measure audio programme loudness and true-peak audio level. EBU R 128 and EBU Tech 3341/3342.
41. R. Bristow-Johnson. "Cookbook formulae for audio EQ biquad filter coefficients." S. H. Linkwitz. "Active crossover networks for noncoincident drivers." JAES 24(1), 1976.
42. V. Zavalishin. The Art of VA Filter Design (TPT/SVF), 2012–2018; A. Simper, "Linear trapezoidal integrated SVF" technical papers, Cytomic.